

Day 4

Strings & Exception Handling in Java

Complete Interview-Ready Notes

Internal Working • Real-World Examples

Complete Code • 35 Interview Questions

Coding Problems • AI-Style Questions • Mock Interview

Table of Contents

- 1** Strings — Internal Working & Memory Model
- 2** String Pool, Immutability, Real-World Relevance
- 3** StringBuilder vs StringBuffer vs String
- 4** Important String Methods (with code)
- 5** Exception Handling — Internal Working
- 6** try / catch / finally / throw / throws
- 7** Custom Exceptions & Real-World Use
- 8** Checked vs Unchecked Exceptions
- 9** Coding Problems (6 solved, with dry-run notes)
- 10** 35 Interview Questions & Answers
- 11** AI / Scenario-Based Interview Questions
- 12** Mock Interview Transcript

Part 1 — Strings

1.1 Internal Working: The String Pool

Every Java application has a special memory region inside the heap called the **String Constant Pool** (SCP). Since Java 7, this pool lives in the heap (earlier it was in PermGen). When you create a String using a literal, the JVM first checks whether an identical value already exists in the pool.

How JVM decides pooling

`String a = "java";` → JVM looks in the pool. Not found → creates it and stores the reference in `a`.

`String b = "java";` → JVM finds `"java"` already in pool → `b` just points to the same object.
No new object created.

```
String a = "java";
String b = "java";
String c = new String("java"); // bypasses pool lookup, creates new heap object
String d = c.intern();         // forces JVM to return pooled reference

System.out.println(a == b);    // true (same pool object)
System.out.println(a == c);    // false (c is a separate heap object)
System.out.println(a == d);    // true (intern() returns pooled reference)
System.out.println(a.equals(c)); // true (value comparison)
```

Memory diagram (conceptual)

```
Heap
├── String Pool
│   ├── "java" ← a
│   │   └── b
│   │   └── d (after intern())
└── Regular Heap Space
    ├── String object("java") ← c (new String(...))
```

1.2 Why String is Immutable

The internal character data of a `String` (a `final char[]` in older JDKs, `final byte[]` since Java 9's compact strings) can never change after construction. Any method that looks like it "modifies" a string (`concat`, `replace`, `substring`, `toUpperCase`) actually returns a brand-new `String` object.

Real-world reason this matters

Because Strings are immutable, the JVM can safely let thousands of references point at one pooled `"admin"` or `"SELECT * FROM users"` object without fear that one part of the program mutates it and silently breaks another part — this is exactly why Strings are used for **HashMap keys**, **database queries**, **file paths**, and **class loading identifiers** (`Class.forName("com.foo.Bar")`). If Strings were mutable, changing a class name string after loading could corrupt the classloader cache.

```
String s = "abc";
s.concat("def");           // creates "abcdef" object, but discards it – no reference kept
System.out.println(s);    // prints "abc", NOT "abcdef"

s = s.concat("def");       // now s is reassigned to the new object
System.out.println(s);    // prints "abcdef"
```

Four core reasons Strings are immutable

1. **String Pool safety** — sharing one object across many references only works if nobody can mutate it.
2. **Thread safety** — an immutable object can be freely shared across threads with zero synchronization.
3. **HashMap/HashSet key integrity** — `hashCode()` is cached at creation; if the value could change, the object would "get lost" in its own hash bucket.
4. **Security** — network URLs, file paths, and reflection class names are passed as Strings; immutability prevents a downstream method from tampering with them after a security check has already validated them (a classic "TOCTOU" attack vector otherwise).

1.3 StringBuilder vs StringBuffer vs String

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized methods)
Performance	Slow for repeated concatenation	Fastest	Slower than StringBuilder (sync overhead)
Introduced	JDK 1.0	JDK 1.5	JDK 1.0
Use case	Fixed / rarely-changing text	Single-threaded heavy string building	Multi-threaded shared buffer

Real-world example

Building a large JSON or SQL string inside a loop (e.g. a report generator writing thousands of rows) should always use `StringBuilder` — using `String` concatenation in a loop of 10,000 iterations creates ~10,000 throwaway objects and can visibly slow down a service under load. `StringBuffer` is now rarely chosen deliberately; most "thread-safe buffer" needs are instead solved with `StringBuilder` + explicit synchronization, or by not sharing the builder across threads at all.

```
// Why loop concatenation with String is bad
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i;    // creates a new String object EVERY iteration — O(n^2) overall
}

// Correct approach
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);  // mutates internal buffer — O(n) overall
}
String result2 = sb.toString();
```

How StringBuilder grows internally

Internally, `StringBuilder` wraps a resizable `char[]` (or `byte[]` with compact strings) with a default capacity of 16. When you append past capacity, it allocates a new array of roughly $(oldCapacity * 2) + 2$ and copies the old data over — similar to how `ArrayList` grows.

1.4 Important String Methods

```
String s = " Hello World ";

s.length();           // 15
s.trim();              // "Hello World" (removes leading/trailing whitespace)
s.strip();             // same as trim(), but Unicode-aware (Java 11+)
s.toUpperCase();       // " HELLO WORLD "
s.toLowerCase();       // " hello world "
s.substring(2, 7);     // "Hello"
s.indexOf("World");    // 8
s.contains("World");   // true
s.replace("World", "Java"); // " Hello Java "
s.split(" ");          // ["", "", "Hello", "World", "", ""]
s.charAt(2);           // 'H'
s.toCharArray();      // char[] representation
s.equals("hello world"); // false (case-sensitive)
s.equalsIgnoreCase(" hello world "); // true
s.compareTo("Hello");  // lexicographic comparison
String.join("-", "a","b","c"); // "a-b-c"
String.format("Name: %s, Age: %d", "Sam", 25); // formatted string
```

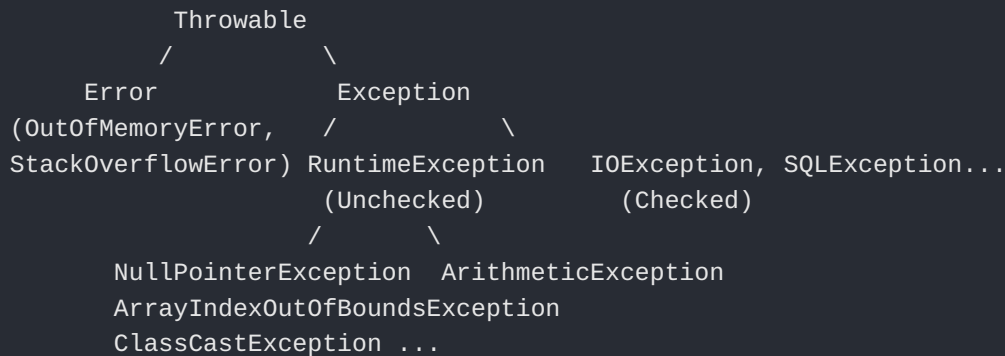
1.5 String Memory Management Summary

Creation style	Where it lives	Garbage collected?
<code>String s = "abc"</code>	String Pool (heap)	Only when pool itself is cleaned up / unreferenced (rare, since literals are usually retained by class metadata)
<code>new String("abc")</code>	Regular heap	Yes, as soon as unreferenced
<code>StringBuilder</code> / <code>StringBuffer</code>	Regular heap	Yes, as soon as unreferenced



Part 2 — Exception Handling

2.1 Internal Working: The Exception Hierarchy



When an exceptional event occurs, the JVM creates an **exception object** containing the type, message, and a snapshot of the call stack (**stack trace**). This object is "thrown" and the JVM starts unwinding the call stack, frame by frame, looking for a matching **catch** block. If none is found anywhere up the call chain, the JVM's default handler prints the stack trace and terminates that thread.

What actually happens in the JVM

Each thread has a call stack of method frames. Throwing an exception does **not** search linearly through code — it uses an internal **exception table** compiled into the bytecode for every method, mapping try-block byte-offset ranges to handler offsets and exception types. The JVM checks this table first in the current frame; if no match, it pops the frame and repeats in the caller — this is why an uncaught exception "bubbles up."

2.2 try / catch / finally

```
try {
    int[] arr = new int[3];
    arr[5] = 10; // throws ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("This always runs – used for cleanup");
}
```

Real-world example

Every database call in production code follows this shape: open a `Connection` in `try`, run the query, catch `SQLException` to log/translate it into a domain error, and close the connection in `finally` (or better, use try-with-resources) so a connection leak never happens even if the query throws.

```
// try-with-resources (preferred modern style – auto-closes resources)
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.executeUpdate();
} catch (SQLException e) {
    log.error("DB operation failed", e);
}
// conn and ps are closed automatically, even if an exception is thrown
```

finally — special cases

- `finally` runs even if `try` or `catch` has a `return` statement.
- `finally` does **not** run if the JVM crashes or `System.exit()` is called inside `try`.
- If both `try` and `finally` have a `return`, the `finally` return silently overrides the `try` return — a classic interview trap.

2.3 throw vs throws

	throw	throws
Purpose	Actually throws an exception instance	Declares a method might throw an exception
Used where	Inside a method body	In the method signature
Followed by	A single exception object	One or more exception class names

```
void withdraw(double amount) throws InsufficientBalanceException {
    if (amount > balance) {
        throw new InsufficientBalanceException("Balance too low for this
withdrawal");
    }
    balance -= amount;
}
```

2.4 Custom Exceptions

Real-world example

In a banking or e-commerce backend, generic exceptions like `Exception` are too vague for the caller to react to programmatically. Teams define domain-specific exceptions —

`InsufficientBalanceException`, `OrderNotFoundException`, `PaymentDeclinedException` — so the calling layer (e.g., a REST controller) can map each one to the correct HTTP status code and error message.

```
// Checked custom exception – caller MUST handle it
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Unchecked custom exception – caller MAY handle it
class InvalidOrderException extends RuntimeException {
    public InvalidOrderException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance = 1000;

    void withdraw(double amt) throws InsufficientBalanceException {
        if (amt > balance) {
            throw new InsufficientBalanceException("Not enough balance!");
        }
        balance -= amt;
    }
}

public class Demo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        try {
            acc.withdraw(5000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }
    }
}
```

2.5 Checked vs Unchecked Exceptions

	Checked	Unchecked
Checked at	Compile time	Runtime
Must declare or handle?	Yes (compiler enforces)	No
Base class	<code>Exception</code> (not <code>RuntimeException</code>)	<code>RuntimeException</code>
Typical cause	External factors (files, network, DB)	Programming bugs (null refs, bad casts)
Examples	<code>IOException</code> , <code>SQLException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code>

Design guideline used in real projects

Use a **checked** exception when the caller can reasonably be expected to *recover* from it (e.g., retry a failed file read). Use an **unchecked** exception when the failure represents a *bug* the caller shouldn't be forced to handle everywhere (e.g., passing a null where it's never valid). Many modern frameworks (Spring) deliberately favor unchecked exceptions to avoid forcing try-catch boilerplate throughout the codebase.



Part 3 — Coding Problems

Problem 1 — Reverse a String

```
public static String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}

// Without built-in reverse() - two-pointer swap, O(n) time, O(n) space
public static String reverseManual(String s) {
    char[] arr = s.toCharArray();
    int i = 0, j = arr.length - 1;
    while (i < j) {
        char temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++; j--;
    }
    return new String(arr);
}
```

Problem 2 — Palindrome Check

```
public static boolean isPalindrome(String s) {
    String clean = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int i = 0, j = clean.length() - 1;
    while (i < j) {
        if (clean.charAt(i) != clean.charAt(j)) return false;
        i++; j--;
    }
    return true;
}

// isPalindrome("A man, a plan, a canal: Panama") -> true
```

Problem 3 — Anagram Check

```
public static boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    char[] a1 = s1.toCharArray();
    char[] a2 = s2.toCharArray();
    Arrays.sort(a1);
    Arrays.sort(a2);
    return Arrays.equals(a1, a2);
}

// O(n log n) due to sort. A frequency-array approach gives O(n).
```

Problem 4 — Character Frequency Count

```
public static Map charFrequency(String s) {
    Map freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    return freq;
}
// charFrequency("hello") -> {h=1, e=1, l=2, o=1}
```

Problem 5 — Remove Duplicate Characters

```
public static String removeDuplicates(String s) {
    LinkedHashSet set = new LinkedHashSet<>();
    for (char c : s.toCharArray()) set.add(c);
    StringBuilder sb = new StringBuilder();
    for (char c : set) sb.append(c);
    return sb.toString();
}
// removeDuplicates("programming") -> "progamin"
```

Problem 6 — Exception-Handling Program (custom + checked/unchecked)

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) { super(msg); }
}

public class ExceptionDemo {

    static void divide(int a, int b) {
        try {
            System.out.println("Result: " + (a / b));
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " + e.getMessage());
        } finally {
            System.out.println("Division attempt finished");
        }
    }

    static void withdraw(double balance, double amt) throws
    InsufficientBalanceException {
        if (amt > balance) throw new InsufficientBalanceException("Not enough
balance!");
        System.out.println("Withdrawal successful");
    }

    public static void main(String[] args) {
        divide(10, 0);
        try {
            withdraw(1000, 5000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }
    }
}
```

Part 4 — 35 Interview Questions & Answers

Strings

1. Why is String immutable in Java?

To support String Pool sharing, thread safety, safe use as HashMap keys, and security in class loading / networking.

2. Difference between == and .equals() for Strings?

`==` compares references (memory addresses); `.equals()` compares actual character content.

3. How many objects are created by `String s = new String("abc")` ?

Up to two: one in the String Pool (if `"abc"` isn't already there) and one on the regular heap for the `new String(...)` call.

4. What does `intern()` do?

Returns the canonical pooled reference for a String's value, adding it to the pool first if not already present.

5. Why are Strings good HashMap keys?

Their hashCode is computed once and cached; since the object can't mutate, the cached hash always stays valid.

6. StringBuilder vs StringBuffer — when do you choose which?

StringBuilder for single-threaded code (faster, no sync overhead); StringBuffer only when multiple threads truly share the same mutable buffer.

7. Why is repeated `+=` concatenation in a loop bad practice?

Each concatenation creates a new String object, making the loop $O(n^2)$ overall instead of $O(n)$ with StringBuilder.

8. Is String thread-safe? Why?

Yes — because it's immutable, multiple threads can read the same String object without any risk of one thread's changes affecting another.

9. What's the default capacity of StringBuilder?

16 characters; it grows automatically (roughly doubling) once that capacity is exceeded.

10. Difference between `substring()` pre- and post-Java 7?

Before Java 7, `substring()` shared the original char array (risking memory leaks by holding a big array alive via a small substring). Since Java 7, it copies only the needed characters into a new array.

11. What are compact Strings (Java 9+)?

An internal optimization where Strings are stored as a `byte[]` with a coder flag (Latin-1 vs UTF-16) instead of always using `char[]`, halving memory for most ASCII-heavy strings.

12. Can you compare Strings with `<` or `>`?

No — Java doesn't overload operators; use `compareTo()` for lexicographic comparison.

13. What does `String.format()` do internally?

Delegates to a `Formatter` object which parses the format specifiers and builds the final String via an internal `StringBuilder`.

14. Difference between `trim()` and `strip()`?

`trim()` removes characters $\leq$ U+0020; `strip()` (Java 11+) is Unicode-aware and correctly handles other whitespace characters too.

15. Is it possible for the String Pool to cause a memory leak?

Rarely, but yes — excessive use of `intern()` on many unique dynamic strings can bloat the pool since pooled strings aren't eligible for GC as long as referenced.

Exception Handling

16. Difference between checked and unchecked exceptions?

Checked exceptions are verified at compile time and must be declared or handled; unchecked (RuntimeException-based) ones are not checked by the compiler.

17. Difference between `Exception` and `Error`?

`Exception` represents recoverable application-level problems; `Error` represents serious JVM-level problems (like `OutOfMemoryError`) that applications generally shouldn't try to catch.

18. Can `finally` ever not execute?

Yes — if `System.exit()` is called inside try/catch, or the JVM crashes, or the thread is forcibly killed.

19. What happens if both try and finally have a return statement?

The finally block's return silently overrides the try block's return value.

20. Can you have try without catch?

Yes, as long as it's paired with a finally block: `try { } finally { }`.

21. Can a catch block catch multiple exception types?

Yes, using the multi-catch syntax: `catch (IOException | SQLException e) { }` (Java 7+).

22. Why should specific exceptions be caught before generic ones?

Because catch blocks are checked top-to-bottom; a broader type like `Exception` placed first would catch everything and make the more specific catch block unreachable (a compile error in Java).

23. Difference between throw and throws?

`throw` actually raises a specific exception instance inside a method body; `throws` just declares in the method signature that the method may propagate that exception type.

24. What is exception chaining?

Wrapping a lower-level exception inside a higher-level one via the constructor's `cause` parameter, so the original stack trace is preserved:

```
throw new ServiceException("failed", originalException).
```

25. Why prefer unchecked exceptions in modern frameworks like Spring?

They avoid forcing try-catch boilerplate at every call site, letting a small number of centralized handlers deal with errors (e.g., a global `@ExceptionHandler`).

26. What's the difference between `final`, `finally`, and `finalize()`?

`final` is a modifier for constants/no-override/no-extend; `finally` is a block that always executes after try/catch; `finalize()` was a (now-deprecated) method the GC could call before reclaiming an object.

27. How does try-with-resources work internally?

Any resource implementing `AutoCloseable` declared in the try's parentheses has its `close()` method called automatically at the end of the block, even if an exception is thrown — the compiler generates the equivalent of a finally block for you.

28. What is a "suppressed exception"?

If both the try block and the automatic `close()` in try-with-resources throw exceptions, the `close()` exception is attached as a "suppressed" exception on the primary one rather than replacing it.

29. Can constructors throw exceptions?

Yes — constructors can throw both checked and unchecked exceptions just like regular methods.

30. What happens if an exception is thrown in a static initializer block?

It's wrapped in an `ExceptionInInitializerError` and the class fails to load.

31. Is it good practice to catch `Throwable`?

Generally no — catching `Throwable` also catches `Error`s like `OutOfMemoryError`, which usually shouldn't be handled as if they were recoverable.

32. How do custom exceptions typically carry extra data (like an error code)?

By adding extra fields and a matching constructor in the custom exception class, e.g. storing an `errorCode` alongside the message for the caller/API layer to use.

33. What's the performance cost of exceptions?

Throwing is relatively expensive mainly due to capturing the stack trace; this is why exceptions should be used for truly exceptional conditions, not routine control flow.

34. Difference between `NullPointerException` and `ClassCastException` ?

NPE occurs when calling a method/field on a null reference; ClassCastException occurs when an invalid downcast is attempted between incompatible types.

35. Should you ever leave a catch block empty?

Almost never — silently swallowing exceptions hides real bugs; at minimum log the exception, or explicitly justify the decision to ignore it in a comment.

Part 5 — AI / Scenario-Based Interview Questions

These are the kind of open-ended, judgment-based questions that AI-driven interview platforms and senior interviewers now favor over textbook definitions.

1. You're processing 1 million rows and building a report string for each — how would you optimize it, and why?

Use a single reused `StringBuilder` instead of String concatenation, since each `+=` would create a new immutable object per row ($O(n^2)$ total cost); also consider writing directly to a `BufferedWriter` / stream instead of holding the entire report in memory.

2. A teammate wrote a custom exception that extends `RuntimeException` for a "user not found" scenario in a REST API. Is that the right choice?

Generally yes in a Spring-style API — unchecked exceptions keep controller code clean, and a global exception handler can map `UserNotFoundException` to a 404 response without every calling method needing a try-catch or throws declaration.

3. Your service intermittently throws `OutOfMemoryError` and a colleague suggests wrapping the whole service in try-catch to "handle" it. What's your take?

`Error`s like OOM usually indicate the JVM itself is in an unstable state; catching it doesn't fix the root cause (likely a memory leak or undersized heap) and can mask the real problem — better to investigate with heap dumps/profiling than blanket-catch it.

4. How would you design exception handling for a payment-processing microservice that calls three downstream APIs?

Wrap each downstream call's checked exceptions into a domain-specific unchecked exception (e.g., `PaymentGatewayException`) with the original exception chained as the cause, so the caller only deals with one exception hierarchy while stack traces for debugging are preserved.

5. You need to check if two large strings are anagrams of each other, but the strings could contain Unicode characters beyond basic ASCII. Would your usual char-array sort approach still work?

It works but may not correctly compare characters outside the Basic Multilingual Plane (astral/surrogate-pair characters), since `char` is only 16-bit UTF-16 code units; a more robust solution iterates using `codePoints()` instead of `toCharArray()`.

6. Why might interning every dynamically-generated String in a high-throughput service actually hurt performance?

The String Pool isn't free — every unique interned value stays referenced, growing the pool and increasing GC scan time and memory pressure; interning is only worth it for a small set of frequently-repeated values, not arbitrary dynamic data.

7. How would you explain to a junior developer why catching `Exception` broadly in a loop that processes 1000 independent records is risky?

It can silently swallow programming bugs (like NPEs) alongside expected data errors, making failures invisible; better to catch specific expected exceptions, log/report the rest, and let unexpected ones propagate or be tracked separately per record.

Part 6 — Mock Interview Transcript

A short simulated interview covering both topics conversationally — useful for practicing verbal answers, not just recall.

Interviewer: Let's start simple — what happens in memory when you write `String s = "hello";` twice in two different classes?

Candidate: Both would end up pointing to the same object in the String Constant Pool. The JVM checks the pool before creating a literal, so the second declaration just reuses the existing reference rather than allocating new memory.

Interviewer: And if I instead used `new String("hello")` both times?

Candidate: Then each call creates a distinct object on the regular heap, outside the pool — so `==` between them would be false, even though `.equals()` would still be true since the content matches.

Interviewer: Good. Now, why would you choose `StringBuilder` over string concatenation in a loop?

Candidate: Because `String` is immutable — every concatenation in the loop creates a brand-new object, which is wasteful and turns an $O(n)$ loop into effectively $O(n^2)$ work. `StringBuilder` mutates an internal buffer in place, so it stays $O(n)$.

Interviewer: Let's move to exceptions. Explain checked versus unchecked with a real example you've used.

Candidate: Checked exceptions, like `IOException`, are things the compiler forces you to handle or declare — usually external failures like a missing file. Unchecked ones, like a custom `InsufficientBalanceException` if I extend `RuntimeException`, don't require that — I'd typically use unchecked for domain errors in a service layer so I'm not forced to add throws everywhere, and instead handle them centrally, like in a global exception handler in a Spring app.

Interviewer: If your `try` and `finally` blocks both have return statements, what gets returned?

Candidate: The `finally` block's return wins — it silently overrides whatever the `try` block was about to return. It's a common gotcha, and honestly it's a sign the code should be restructured, since relying on that behavior is confusing.

Interviewer: Last one — when would you write a custom exception instead of just throwing a generic `RuntimeException`?

Candidate: When the calling code needs to react differently based on the specific failure — for example, in an e-commerce order flow, catching `OrderNotFoundException` specifically to return a 404, versus `PaymentDeclinedException` to return a 402. A generic `RuntimeException` loses that distinction and forces the caller to inspect the message string, which is fragile.